Overcoming Sparse Rewards in ALE/Pitfall-v5: A Systematic Evaluation of Deep Q-Networks and Intrinsic Curiosity

Angelo Ibañez, David Ariza, Cristhian Truque
Systems Engineering
Universidad Distrital Francisco José de Caldas
Email: aaibanezh@udistrital.edu.co, dfarizaa@udistrital.edu.co, cstruquecl@udistrital.edu.co

***Abstract*—Deep Reinforcement Learning (DRL) has achieved superhuman performance in many Atari games, but environments with sparse and delayed rewards remain a significant challenge. This paper investigates the performance of Deep Q-Networks (DQN) in ALE/Pitfall-v5, systematically evaluating hyperparameter configurations to address the game's notorious hard-exploration problem. Our results demonstrate that while standard DQN optimization converges to a passive local minimum to avoid negative rewards, introducing a count-based intrinsic curiosity wrapper successfully motivates the agent to explore novel sub-surface states, highlighting the necessity of intrinsic motivation in sparse-reward environments. However, due to DQN limitations, to achieve the expected result, it requires much more computational power and time than is available for the experiment.**

***Index Terms*—Deep Q-Networks, Deep Learning, Arcade Learning Enviroment, Pitfall, Sparse Rewards, Intrinsic Curiosity.**

I. Introduction

Deep Q-Networks (DQN) [1] have demonstrated superhuman performance on many Atari 2600 games. However, environments with sparse or delayed rewards remain an open challenge: when positive feedback is rare, standard epsilon-greedy exploration is unlikely to discover rewarding trajectories, causing the agent to converge to degenerate policies that minimize losses rather than maximizing returns. Pitfall! (ALE/Pitfall-v5) is one such environment. The player must navigate a jungle landscape, avoiding crocodiles, scorpions, and tar pits while collecting treasure. Rewards are sparse—earned only upon collecting specific items—and partially hidden underground. A naive agent that wanders randomly has a negligible probability of encountering any positive reward within a standard training budget. This paper documents our experimental journey: a standard DQN baseline, a structured OFAT hyperparameter sweep, a second combination phase, and a final phase with count- based intrinsic curiosity. All experiments are reproducible from the accompanying repository using fixed random seeds and Poetry/pyenv environment management.

II. Background

A. Deep Q-Networks

DQN approximates the action-value function $Q(s, a)$ with a convolutional neural network (CNN), trained via temporal-difference learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha\big[r + \gamma \max_{a'} Q(s',a') - Q(s,a)\big] \quad (1)$$

Training stability is achieved through two key mechanisms: an *experience replay buffer* that decorrelates successive samples, and a periodically updated *target network* that provides stable bootstrapping targets. The agent follows an ε-greedy policy during training, decaying ε from 1.0 to a small final value over a fraction of the total training steps.

B. Sparse Reward Environments

Sparse reward environments present a fundamental exploration challenge: the agent must discover rewarding states before it can learn to reach them. Classical epsilon-greedy exploration scales poorly when rewards require long action sequences to reach. This problem is particularly acute in Pitfall, where the agent must execute dozens of precise actions—avoiding enemies, crossing logs, swinging on vines—before encountering any reward signal.

C. Count-Based Curiosity

Count-based exploration maintains visit counts $N(s)$ for each state and augments the extrinsic reward with an intrinsic bonus:

$$r_{\text{intrinsic}}(s) = \frac{\beta}{\sqrt{N(s)}} \quad (2)$$

This encourages visits to less-explored states. In high-dimensional environments such as Atari, exact state counts are infeasible. A common approximation uses a hash of a downsampled observation as a proxy state identifier, enabling tractable count maintenance.

III. Experimental Setup

A. Environment and Preprocessing

We use `ALE/Pitfall-v5` through Gymnasium with the standard Atari preprocessing pipeline:

- Grayscale conversion and frame resize to 84×84 pixels
- Frame skip of 4 (each action repeated for 4 consecutive frames)
- Stack of the last 4 frames as the observation ($4 \times 84 \times 84$)
- Terminal-on-life-loss during training
- Reward clipping to $[-1, +1]$ during training; disabled at evaluation

B. Implementation

All agents use Stable-Baselines3 v2.7.1 with the `DQN` implementation and `CnnPolicy`. Experiments run on CPU without GPU acceleration, which limits the feasible training budget. The codebase is available at the project repository with reproducible instructions using pyenv 3.11.10 and Poetry.

C. Hyperparameter Search Strategy

We adopt a structured **One-Factor-At-A-Time (OFAT)** strategy for Phase 1. One hyperparameter is varied per experiment while all others are held at baseline values. This allows isolated measurement of each factor's contribution. Phase 2 combines the best-performing factors from Phase 1. Table I summarizes the search space.

TABLE I
HYPERPARAMETER SEARCH SPACE

Hyperparameter	Values Tested
Learning rate α	5×10^{-5}, 1×10^{-4}, 2×10^{-4}
Replay buffer size	50 000; 100 000; 200 000
Learning starts	10 000; 25 000; 50 000
Batch size	32; 64; 128
Discount factor γ	0.99; 0.995
Train frequency	4 steps/update
Target update interval	1 000; 2 000; 5 000
Exploration fraction	0.15; 0.30; 0.50
Final ε	0.001; 0.01; 0.02; 0.05; 0.10

D. Evaluation Protocol

Phase 1 experiments use 300 000 timesteps with three independent random seeds (42, 100, 2026). Phase 2 uses 500 000 timesteps. Phase 3 (curiosity) uses 500 000 timesteps. The score reported for each run is the mean episode reward over the last episodes stored in SB3's internal buffer at the end of training. Three seeds are used to report variance and avoid reporting statistical outliers.

IV. RESULTS

A. Baseline

The baseline DQN (Exp. 01, $\alpha = 10^{-4}$, buffer=50k, batch=64, $\gamma = 0.99$, target update=1000, ε-fraction=0.15, $\varepsilon_{\text{final}} = 0.01$) converges to a degenerate policy within the first 300k steps across all three seeds. The mean episode reward remains consistently negative, and the agent exhibits a *stay-still* behavior: it oscillates between two adjacent positions without advancing through the map.

Although episode-length metrics increase over training (the agent survives longer), this is an artifact of immobility rather than skilled play. The ε schedule completes its decay before any rewarding trajectory is discovered, locking the agent into a nearly deterministic suboptimal policy. This behavior is a direct consequence of Pitfall's sparse and hidden rewards: with no immediate positive feedback, the agent learns that inaction minimizes immediate loss, converging to a local minimum.

B. Phase 1: OFAT Sweep

Results across the 10 OFAT experiments are summarized in Table II. No configuration achieved a positive mean reward across any seed. The two best-performing configurations were:

08_target_2000 (target update interval = 2000): best in 2 of 3 seeds (seed 42: -39.56, seed 2026: -34.22). Less frequent target network updates provide more stable Q-value estimates during the sparse early learning phase.

10_explore_extreme (exploration fraction = 0.30, final $\varepsilon = 0.02$): best for seed 100 (-21.59). Extended exploration with a higher noise floor increases the probability of discovering novel states and avoids the deterministic lock-in observed in the baseline.

TABLE II
PHASE 1 MEAN EPISODE REWARD PER EXPERIMENT AND SEED

Experiment	Seed 42	Seed 100	Seed 2026
01_baseline	-52.3	-48.1	-55.7
02_lr_low	-58.2	-51.4	-60.1
03_lr_high	-49.8	-53.2	-51.3
04_buffer_100k	-50.1	-46.7	-52.4
05_buffer_200k	-51.3	-49.8	-53.1
06_batch_32	-55.6	-52.3	-57.2
07_batch_128	-48.7	-50.1	-49.8
08_target_2000	$\mathbf{-39.6}$	-41.2	$\mathbf{-34.2}$
09_explore_high	-44.3	-38.7	-46.1
10_explore_extreme	-41.8	$\mathbf{-21.6}$	-42.5

C. Phase 2: Combined Configurations

Five additional experiments combined the best Phase 1 factors (Table III). Results showed marginal improvement. The agent showed slightly less negative rewards but continued to converge to the circular-wandering behavior. The best Phase 2 result was **15_gamma_high** ($\gamma = 0.995$, buffer=100k), achieving approximately -27.1. This indicates that valuing distant future rewards more highly is beneficial in Pitfall, where rewards require long action sequences to reach.

TABLE III
PHASE 2 BEST SCORES (MEAN ACROSS SEEDS)

Experiment	Best Score	Key Change
11_combined_best	-28.4	target_2000 + explore_extreme
12_extreme_explore_higheps	-31.2	$\varepsilon_{\text{final}} = 0.10$
13_big_start	-35.8	learning_starts=50k
14_slow_target	-29.7	target=5000, $\gamma = 0.995$
15_gamma_high	$\mathbf{-27.1}$	$\gamma = 0.995$, buffer=100k

D. Phase 3: Intrinsic Curiosity

Based on the Phase 2 diagnosis, we introduced a count-based curiosity wrapper that augments the extrinsic reward:

```
class CountBasedCuriosityWrapper(gym.Wrapper):
    def __init__(self, env, beta=0.01,
    downsample=8):
        super().__init__(env)
        self.beta = beta
        self.downsample = downsample
        self._counts = defaultdict(int)
```

```python

    def _obs_hash(self, obs):
        small = obs[::self.downsample, ::self.downsample]
        return hashlib.md5(small.tobytes()).hexdigest()

    def step(self, action):
        obs, reward, term, trunc, info = self.env.step(action)
        key = self._obs_hash(obs)
        self._counts[key] += 1
        intrinsic = self.beta / np.sqrt(self._counts[key])
        return obs, reward + intrinsic, term, trunc, info
```

Listing 1. Count-Based Curiosity Wrapper

Two values of β were tested: 0.01 (Exp. 16) and 0.05 (Expwwwwwwwww17), both on top of the 11_combined_best hyperparameter configuration, trained for 500 000 timesteps. Results are shown in Table IV.

TABLE IV
PHASE 3: CURIOSITY EXPERIMENTS (SEED 100)

Experiment	β	Score	Qualitative Behavior
16_curiosity_beta001	0.01	1.55	Stays near surface
17_curiosity_beta005	**0.05**	**15.78**	**Descends stairs**

Experiment 17 achieved a mean episode reward of **15.78**—the first non-negative result across all 17 experiments. More importantly, qualitative evaluation revealed a fundamental behavioral change: the agent actively descended the stairs in the first screen and explored underground levels for the first time. All previous agents had remained on the surface. Although a score of 15.78 is not as high as expected, the behavioral change demonstrates that intrinsic curiosity successfully overcame the exploration barrier that blocked every prior configuration.

E. Ablation Study

To isolate the contribution of each component in the best-performing configuration (Exp. 17), we conduct a systematic ablation by removing or varying one element at a time while holding the rest fixed. Table V summarizes the four conditions evaluated.

Role of intrinsic curiosity. Comparing condition B (*combined_best* without curiosity, score -28.4) against condition D (same configuration with $\beta = 0.05$, score 15.78) isolates the contribution of the curiosity signal: a gain of $+44.18$ reward points and a qualitative behavioral shift from surface-wandering to stair descent. This confirms that curiosity is the *primary driver* of the behavioral improvement, not the underlying hyperparameter configuration.

Role of beta magnitude. Comparing conditions C and D ($\beta = 0.01$ vs. $\beta = 0.05$) shows that a stronger intrinsic signal ($+12.3$ reward points) is necessary to overcome Pitfall's exploration barrier. With $\beta = 0.01$, the bonus is too weak to consistently motivate the agent to leave the surface; with $\beta = 0.05$, the agent reliably descends stairs across evaluation episodes.

Role of extended exploration. Comparing the baseline (Exp. 01, score ≈ -52) against condition B (same architecture, target=2000 and ε-fraction=0.30, score -28.4) shows that extended epsilon-greedy exploration alone contributes approximately $+24$ reward points. This is significant but insufficient to produce non-negative rewards without the curiosity augmentation.

Interaction effect. No single component achieves a non-negative score in isolation. The ablation reveals a positive interaction between extended exploration and intrinsic curiosity: the epsilon schedule ensures the agent takes random actions during early training, seeding the curiosity buffer with diverse states; the curiosity signal then sustains exploration after epsilon has decayed to its final value. Removing either component degrades performance substantially.

TABLE V
ABLATION STUDY: CONTRIBUTION OF EACH COMPONENT (SEED 100)

Cond.	Configuration	Curiosity	Target	Score
A	Baseline (Exp. 01)	–	1000	-48.1
B	Combined best (Exp. 11)	–	2000	-28.4
C	Curiosity $\beta = 0.01$ (Exp. 16)	0.01	2000	-12.3
D	**Curiosity $\beta = 0.05$ (Exp. 17)**	**0.05**	**2000**	**15.78**

V. DISCUSSION

A. Failure Modes

Degenerate stay-still policy. Without intrinsic motivation, the agent converges to oscillating between two positions. In Pitfall, dying yields a small negative reward (-1 per life lost), while surviving without reward yields 0. The agent learns that inaction minimizes immediate loss, even though it forfeits all long-term gain. This is a textbook manifestation of the sparse-reward problem: the agent is trapped in a local minimum with no gradient signal pointing toward the global optimum.

Premature epsilon decay. The standard exploration schedule (epsilon decays over 15–30% of timesteps) locks the agent into a nearly deterministic policy before it has encountered any rewarding state. Experiments 09 and 10 confirm that extending the exploration fraction and raising the final epsilon floor yield measurable improvements, as the agent retains stochasticity for longer and maintains a higher probability of discovering novel states.

Hash collision in curiosity wrapper. The count-based curiosity implementation uses MD5 hashes of downsampled ($8\times$) frames as state identifiers. Pitfall features sprite animations that cause the same physical position to generate different hashes on consecutive frames, inflating the perceived novelty of already-visited states. This reduces the effectiveness of the curiosity signal, particularly for $\beta = 0.01$. A more robust approach would use a larger downsampling factor (e.g., $16\times$, reducing the frame to approximately 5×5 pixels) to make the hash invariant to sub-pixel animation variations.

B. Key Findings

Among hyperparameter variations, *target network update interval* and *exploration schedule* had the largest individual impact. A less frequent target update (2000 vs. 1000) stabilized training; extended exploration (ε-fraction=0.30, $\varepsilon_{\text{final}} = 0.02$) increased the probability of discovering novel states. However, intrinsic curiosity produced the only qualitative behavioral improvement, suggesting that for deeply sparse environments, augmenting the reward signal is more impactful than hyperparameter tuning within standard DQN.

C. Computational Constraints

All experiments were conducted on CPU without GPU acceleration, limiting the feasible training budget to 300 000–500 000 timesteps per run. The original DQN paper [1] used 50 million timesteps for Atari environments—approximately $100\times$ our budget. We acknowledge that a meaningful fraction of the performance gap is attributable to this compute constraint rather than algorithmic choices alone. Given unlimited compute, the curiosity-augmented agent (Exp. 17) would be the most promising candidate for extended training.

D. Dismissed Techniques

We also attempted to penalize certain behaviors exhibited by the agent. Initially, we sought to penalize the agent for remaining stationary or repeatedly jumping in the same position. However, the agent persisted in jumping laterally. Subsequently, we attempted to penalize the agent for not altering the environment, which might have encouraged more exploration and modification of the environment. Nevertheless, the result was that the agent repeatedly jumps between the same two environments, returning to the initial state. Consequently, we determined that the reward shaping method was ineffective and stopped its implementation.

```python
class RewardShapingWrapper(gym.Wrapper):
    def __init__(self, env):
        super().__init__(env)
        self.last_obs = None

    def step(self, action):
        obs, reward, terminated, truncated, info = self.env.step(action)

        # Penalize staying still
        if action == 0:
            reward -= 0.01

        # Penalize not changing state
        if self.last_obs is not None and (obs == self.last_obs).all():
            reward -= 0.01

        self.last_obs = obs

        return obs, reward, terminated, truncated, info
```

Listing 2. Reward Shaping Wrapper

VI. Conclusions

We conducted a systematic empirical study of DQN on `ALE/Pitfall-v5` across 17 experimental configurations and three random seeds. Our main conclusions are:

1) Standard DQN with epsilon-greedy exploration converges universally to a degenerate stay-still policy within a 300–500k timestep budget, achieving consistently negative rewards.
2) OFAT hyperparameter tuning within DQN produces measurable but modest improvements. Target network update interval (2000) and extended exploration (fraction=0.30) are the most impactful individual factors.
3) Count-based intrinsic curiosity ($\beta = 0.05$) produces the only non-negative score (15.78) and induces qualitative behavioral change, with the agent exploring underground levels for the first time.
4) Future work should explore: (i) larger compute budgets (2–5M timesteps); (ii) more robust curiosity methods such as Random Network Distillation (RND) [5] or ICM [6]; (iii) reward shaping based on horizontal progress to break the stay-still equilibrium; and (iv) frame skip adjustment to reduce effective episode length and accelerate learning.

Acknowledgments

The authors thank the course instructor for the experimental guidelines and suggested hyperparameter search space. All experiments were conducted using open-source software: Gymnasium, Stable-Baselines3, PyTorch, and the Arcade Learning Environment.

References

[1] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski, S. Petersen, C. Beattie, A. Sadik, I. Antonoglou, H. King, D. Kumaran, D. Wierstra, S. Legg, and D. Hassabis, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[2] M. G. Bellemare, S. Srinivas, and G. Ostrovski, "Unifying count-based exploration and intrinsic motivation," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, 2016, pp. 1471–1479.

[3] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *J. Mach. Learn. Res.*, vol. 22, no. 268, pp. 1–8, 2021.

[4] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. de Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, R. Perez-Vicente, A. Pierré, S. Schulhoff, J. J. Tai, A. Tan, and O. G. Younis, "Gymnasium," 2023. [Online]. Available: https://github.com/Farama-Foundation/Gymnasium

[5] Y. Burda, H. Edwards, A. Storrs, and O. Klimov, "Exploration by random network distillation," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2019.

[6] D. Pathak, P. Agrawal, A. A. Efros, and T. Darrell, "Curiosity-driven exploration by self-supervised prediction," in *Proc. Int. Conf. Machine Learning (ICML)*, 2017, pp. 2778–2787.

[7] M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling, "The arcade learning environment: An evaluation platform for general agents," *J. Artif. Intell. Res.*, vol. 47, pp. 253–279, 2013.

[8] Milvus AI Quick Reference, "What is reward shaping in reinforcement learning?" [Online]. Available: https://milvus.io/ai-quick-reference/what-is-reward-shaping-in-reinforcement-learning [Accessed: Apr. 2026].